

Chapter 1 — REST API & ASP.NET Core Web API: Foundations

1. Understanding REST (Representational State Transfer)

REST is an **architectural style** used to design **HTTP-based distributed systems**. It enables applications such as web clients, mobile apps, and IoT devices to communicate with servers — most commonly using **JSON-formatted data**.

Why REST Exists

Modern applications require:

- Support across **multiple client platforms**
- **Fast and lightweight** communication
- **Scalability**
- **Independence from programming language or platform**

REST allows this because it runs completely over the **HTTP protocol**.

2. Core REST Concepts and Entities

Term	Meaning
Resource	The data being exposed (e.g., Student, Member, Product)
Endpoint (URI)	The URL used to access a resource
HTTP Method	Defines what operation to perform
Headers	Carry extra metadata such as format or authentication
Data Format	Usually JSON (XML is also supported)

3. HTTP Methods Used in REST

Method	Purpose
GET	Retrieve data
POST	Insert a new record
PUT	Update an existing record
DELETE	Remove a record

These map naturally to CRUD.

4. What Is ASP.NET Core Web API?

ASP.NET Core Web API is a **framework for building HTTP-based services** that return **JSON by default** and can be consumed by:

- Web applications
- Mobile apps
- Desktop applications
- IoT devices
- Cloud systems

Key Characteristics

- Uses HTTP protocol
 - Cross-platform (Windows, Linux, macOS)
 - Ideal for CRUD-based applications
-

5. Why Do We Need Web API?

Web API standardizes data sharing because:

- Modern systems run on **multiple platforms**
 - Most frontends expect **JSON data**
 - A centralized API ensures **consistent output**
-

6. Basic Web API Architecture Flow

1. **Client** sends HTTP request
 2. Request reaches **API Controller**
 3. Controller interacts with **business/data layer**
 4. **JSON response** is returned
-

7. CRUD Operations and HTTP Verbs in Web API

CRUD Operation	HTTP Verb	Purpose
Create	POST	Insert record
Read	GET	Retrieve record(s)
Update	PUT	Modify record
Delete	DELETE	Remove record

8. Model Class Example

```
public class Members
{
    public int MemberId { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Address { get; set; }
}
```

9. Business Layer Contract – Repository Interface

```
public interface IMembers
{
    List<Members> GetAllMember();
    Members GetMember(int id);
    Members Add(Members member);
    Members Update(int id, Members memberChanges);
    Members Delete(int id);
}
```



10. Dependency Injection Registration

```
builder.Services.AddScoped<IMembers, MembersRepository>();
```

This ensures **loose-coupling**.

11. API Controller — Core Implementation

```
[Route("api/[controller]")]
[ApiController]
public class MemberController : ControllerBase
{
    private readonly IMembers member;

    public MemberController(IMembers mem)
    {

```

```

        member = mem;
    }

    [HttpGet]
    public IEnumerable<Members> Get() =>
    {
        member.GetAllMember();
    }

    [HttpGet("{id}")]
    public Members Get(int id) =>
    {
        member.GetMember(id);
    }

    [HttpPost]
    public Members Post([FromBody] Members memberpost) =>
    {
        member.Add(memberpost);
    }

    [HttpPut("{id}")]
    public Members Put(int id, [FromBody] Members memberput) =>
    {
        member.Update(id, memberput);
    }

    [HttpDelete("{id}")]
    public void Delete(int id) =>
    {
        member.Delete(id);
    }
}

```



12. Routing Examples

URL	Meaning
/api/member	Get all members
/api/member/1	Get member having ID = 1
/api/member (POST)	Insert member
/api/member/1 (PUT)	Update member
/api/member/1 (DELETE)	Delete member

13. Attribute Routing with Parameters

```
[HttpGet("{id:int}")]
```

```
public Members Get(int id)
```

```
[HttpGet("{name}")]
```

```
public IEnumerable<Members> Get(string name)
```

To avoid ambiguity, **specify route constraints** such as {id:int}.

14. Additional Routing Patterns

```
[HttpGet("Address/{address}")]
```

```
public IEnumerable<Members> GetAddress(string address)
```

15. ApiController Attribute — Key Benefits

[ApiController] enables:

- Mandatory **attribute-based routing**
- Automatic **400 Bad Request** for validation errors
- Automatic **model binding**
- Binding source inference

Example:

```
[HttpPost]
```

```
public IActionResult Test([FromForm] Model model)
```



1. Why .NET Core Was Introduced

Limitations of the Old .NET Framework

Traditional .NET Framework had some critical restrictions:

- It **ran only on Windows**
- Different platforms used **different APIs**
 - Desktop
 - Windows Store
 - Phone
 - Web
- Framework was **installed system-wide**
 - Updating framework could break existing apps
 - Version conflicts were common
- Not optimized for **modern, cloud-driven, microservices architectures**

Requirements of Modern Applications

Modern software must:

- Run across **multiple operating systems**
- Support **web, cloud, microservices, containers, mobile backends**
- Deliver **high performance**
- Be **lightweight and modular**
- Remain **open-source and community-driven**

Main Objectives of .NET Core

.NET Core was built to address these requirements.

It is:

- **Open-source**
- **Cross-platform**
- **High-performance**
- **Modular and lightweight**

- **Cloud-ready & microservices-friendly**
 - Able to run **from data centers to small devices**
-

2. Key Characteristics of .NET Core

1. Open-Source Development

- Maintained by **Microsoft + community**
- Hosted on **GitHub**
- Licensed under **MIT & Apache 2**
- Major repositories include:
 - Roslyn compiler → github.com/dotnet/roslyn
 - Runtime → github.com/dotnet/runtime
 - SDK → github.com/dotnet/sdk
 - ASP.NET Core → github.com/dotnet/aspnetcore

This ensures transparency and collaboration.

2. Cross-Platform Execution

.NET Core runs on:

- Windows
- Linux
- macOS

The **same code executes identically** on all platforms.

3. Architecture & Hardware Support

Supported processor architectures:

- x86
- x64
- ARM

This allows deployment across servers, desktops, and embedded devices.

4. Types of Applications You Can Build

Using .NET Core, one can develop:

- Web applications & Web APIs
 - Desktop applications
 - Cloud-based applications
 - Mobile backends
 - IoT systems
 - Microservices
 - Machine-learning workloads
 - Games
-

5. Multi-Language Support

Languages supported include:

- C#
- F#
- Visual Basic

Popular IDEs include:

- Visual Studio
 - Visual Studio Code
 - Sublime Text
 - Vim
-



6. Modular (NuGet-Based) Architecture

Key concepts:

- Framework features are shipped as **NuGet packages**
- Applications reference **only what they need**
- This results in:
 - Lower memory footprint
 - Better performance
 - Clean deployment

Default core package:

Microsoft.NETCore.App

7. CLI Tools (Command-Line Interface)

CLI supports development lifecycle:

dotnet new

dotnet build

dotnet run

dotnet publish

Used for automation, scripting, and DevOps pipelines.

8. Flexible Deployment Models

Deployment options:

- User-wide deployment
- System-wide deployment
- **Self-contained deployment**
- **Docker container execution**

This supports both enterprise and cloud environments.

9. Compatibility via .NET Standard

.NET Standard enables **code sharing across platforms**, including:

- .NET Framework
 - Xamarin / Mono
 - .NET Core
-

3. .NET Core Versions

Version	Status
.NET Core 3.1	Long-Term Support (LTS)
.NET Core 2.1	Past LTS support
.NET 5+	Unified platform replacing .NET Core

Key notes:

- **.NET Core 3.x apps run only on .NET Core**
- **.NET Core 2.x apps may also run on .NET Framework**

4. Internal Composition of .NET Core

.NET Core consists of **five major components**:

1. CLI Tools

Used for development, execution, and deployment.

2. Roslyn Compiler

Compiles:

- C#
- Visual Basic

into **Intermediate Language (IL)**.

3. CoreFX

Base Class Libraries, including:

- Collections
- IO
- Networking
- Security
- Data structures
- Threading utilities



4. CoreCLR

The runtime engine that:

- Executes IL
 - Uses **JIT (Just-In-Time) Compilation**
 - Manages **Garbage Collection**
 - Handles **threading and memory**
-

5. Hosting & Execution Model

Used by:

- ASP.NET Core

- Console Applications
-

5. Installing .NET Core — Two Options

1. Install Visual Studio
2. Install only .NET Core SDK / Runtime

Difference Between Runtime & SDK (Very Important)

Component	Purpose
.NET Core Runtime	Only for executing apps
.NET Core SDK	Used for development — includes Runtime + build tools

Therefore:

- Developers install **SDK**
 - Production servers may install **Runtime only**
-

6. Creating & Running a .NET Core App

Verify installation:

dotnet --version

This confirms SDK presence.

7. ASP.NET Core Application Startup Flow (Critical Theory)

Every ASP.NET Core application:

- Begins as a **Console Application**

Entry Point

Program.cs → Main()

Inside Main():

1. A **Web Host** is created
 2. Host is **built**
 3. Host is **run**
 4. Application starts listening for **HTTP Requests**
-

CreateDefaultBuilder()

This method configures:

- Logging

- Configuration Providers
 - Kestrel Web Server
 - Hosting defaults
 - Startup class reference
-

8. Role of Startup Class

The **Startup class** has two essential methods:

1. ConfigureServices()

Purpose:

- Register dependencies & services
- Acts as **Dependency Injection Root**

Examples:

AddControllers() → Web API

AddMvc() → MVC + Views

AddRazorPages() → Razor Pages

2. Configure()

Defines **request processing pipeline** using middleware, including:

- UseRouting()
- UseAuthorization()
- UseEndpoints()

These determine:

- URL-to-controller mapping
 - Request handling sequence
-

9. Example — Basic Web API Controller

[ApiController]

[Route("[controller]")]

public class WeatherForecastController : ControllerBase

{

 [HttpGet]

```
public IEnumerable<WeatherForecast> Get()
{
    ...
}
}
```

Key Observations

- [ApiController] → Enables API-specific behavior
 - [Route] → Defines route template
 - [HttpGet] → Maps GET request
 - Return value is automatically serialized to **JSON**
 - Uses System.Text.Json
-



Chapter 3 — Controller Action Return Types, HTTP Status Codes & Best-Practice Patterns in ASP.NET Core Web API

1. Purpose of a Controller Action

An ASP.NET Core Web API controller action **receives a request and returns a response**.

A response may include:

- Data (JSON by default)
- HTTP status code
- Response headers
- Error details (if any)

ASP.NET Core supports multiple return styles depending on the requirement.

2. Returning Primitive & Complex Types

Example — Primitive Return

```
[HttpGet("Name")]
public string GetName()
{
    return "Return from GetName";
}
```



Example — Complex Object Return

```
[HttpGet("Details")]
public Employee GetEmployeeDetails()
{
    return new Employee { Id = 1001, Name = "Anurag", Age = 28 };
}
```

Behavior

- ASP.NET Core automatically **serializes returned objects to JSON**
- Response header contains:

Content-Type: application/json

When to Use

Use primitive/complex return type **only when the method always returns success data and no alternate status code is required**.

3. Returning Collections

```
[HttpGet("All")]
```

```
public IEnumerable<Employee> GetAllEmployees()
{
    return new List<Employee> { ... };
}
```

Good when data will always exist.

However, when possible missing data needs handling, use IActionResult or ActionResult.

4. IActionResult — Flexible Return Type

IActionResult is used when **different HTTP responses may be required**.

Example

```
[HttpGet("All")]
```

```
public IActionResult GetAllEmployees()
{
    var employees = GetData();

    if (employees.Any())
        return Ok(employees);    // 200 OK
    else
        return NotFound();        // 404 Not Found
}
```



Advantages

- Highly flexible
- Explicit responses like:
 - Ok()
 - NotFound()
 - BadRequest()
 - Created()
- Clear RESTful intent
- Good for error handling

Disadvantages

- Not strongly typed
- Swagger cannot infer return types unless documented
- Slightly less discoverable

Swagger Documentation Tip

```
[ProducesResponseType(StatusCodes.Status200OK, Type = typeof(List<Employee>))]
```

```
[ProducesResponseType(StatusCodes.Status404NotFound)]
```

5. ActionResult — Strongly Typed + HTTP Support

Combines **strong typing with REST responses**.

Example

```
[HttpGet("{id}")]
```

```
public ActionResult<Employee> GetEmployee(int id)
```

```
{
```

```
    if (id == 0)
```

```
        return NotFound();
```

```
    return new Employee { Id = 1001, Name = "Anurag" };
```

```
}
```



Why Preferred

- Strong typing
 - Better Swagger output
 - Clean readability
 - Industry best-practice
-

6. Specific Result Types

Example using **explicit result classes**:

```
public OkObjectResult GetEmployee(int id)
```

```
{
```

```
    var emp = new Employee { Id = id, Name = "Anurag" };
```

```
    return Ok(emp);
```



```
}
```

These are very explicit but less flexible.

7. Returning void

Returning void results in:

204 No Content

Example:

```
[HttpDelete("{id}")]
```

```
public void DeleteEmployee(int id)
```

```
{
```

```
}
```

Use only when **no response body is needed**.

8. Asynchronous Action Return Types

Task with IActionResult

```
[HttpGet("All")]
```

```
public async Task<IActionResult> GetAllEmployees()
```

```
{
```

```
    var list = await GetDataAsync();
```

```
    if (list.Any())
```

```
        return Ok(list);
```

```
    else
```

```
        return NotFound();
```

```
}
```

Task with ActionResult

```
public async Task<ActionResult<List<Employee>>> GetAllEmployees()
```

```
{
```

```
    var list = await GetDataAsync();
```

```
    return Ok(list);
```

```
}
```

Use for **database calls / external API calls / async operations**.

9. Summary — Which Return Type to Use?

Return Type	Use Case
Primitive / Complex Type	Always returning success data only
ActionResult	Different HTTP codes may be returned
ActionResult	Strong typing + HTTP codes (recommended)
Specific Result Types	Need explicit clarity
void	Response body not required
Task<...>	Async operations

10. HTTP 201 Created — When and Why

Use **201 Created** when a new resource is created (typically POST).

Response must include:

- Status code: **201**
- **Location header** → URL of created resource
- Response body → created object

This aligns with REST standards.

11. Methods That Return 201 Created

Method	Description
Created()	URL must be manually supplied
CreatedAtAction()	URL generated from action
CreatedAtRoute()	URL generated from named route

All three:

- ✓ Return **201 Created**
 - ✓ Add **Location header**
 - ✓ Return object in response
-

12. Created() — Manual URL

[HttpPost]

```
public IActionResult Create()
```

```

{
    var created = new { Id = 1, Version = "1.0" };

    string url = $"https://api.com/values/{created.Id}?version={created.Version}";

    return Created(url, created);
}

```

Developer must ensure URL correctness.

13. CreatedAtAction() — Auto-Generated URL

[HttpPost]

```

public IActionResult Create()
{
    var created = new { Id = 1, Version = "1.0" };

    return CreatedAtAction(
        nameof(GetValue),
        new { id = created.Id, version = created.Version },
        created
    );
}

```



Generates:

Location: /api/Values/1?version=1.0

Recommended because route changes do not break code.

14. CreatedAtRoute() — Named Route

[HttpGet("{id}", Name = "GetValueById")]

```

public IActionResult GetValue(int id)
{
    return Ok($"Value {id}");
}

```

[HttpPost]

```

public IActionResult Create()

```

```
{  
    var created = new { Id = 1 };  
    return CreatedAtRoute(  
        "GetValueById",  
        new { id = created.Id },  
        created  
    );  
}
```

Best when:

- Routes are named
- API versioning is used
- Multiple controllers exist

15. Route Constraints — Prevent Ambiguity

Problem scenario:

/api/employee/123

/api/employee/Atul

Framework cannot decide whether path segment is int or string.

Solution — Apply Constraints

```
[Route("{id:int}")]
```

```
public string Get(int id)
```

```
[Route("{name:alpha}")]
```

```
public string Get(string name)
```

Common Route Constraints

Constraint	Meaning	Example
int	Integer only	{id:int}
bool	Boolean	{flag:bool}
datetime	Date	{d:datetime}
guid	GUID format	{id:guid}
alpha	Letters only	{name:alpha}
min(n)	Minimum numeric value	{id:min(10)}
max(n)	Maximum numeric value	{id:max(100)}
range(x,y)	Number within range	{id:range(10,50)}
length(n)	Exact string length	{code:length(5)}
minlength(n)	Minimum string length	{name:minlength(5)}
maxlength(n)	Maximum string length	{name:maxlength(10)}
regex(expr)	Pattern match	{code:regex(^A\d{3}\$)}

Benefits

- ✓ Eliminates ambiguous route errors
- ✓ Validates input early
- ✓ Improves routing accuracy
- ✓ Reduces manual validation



1. Cross-Origin Resource Sharing (CORS)

Why CORS Is Required

Browsers block requests when a frontend and backend run on **different origins**.

Example:

Component	URL
React App	http://localhost:3000
Web API	http://localhost:5000

Since the **domain/port differs**, the browser blocks the call unless **CORS is enabled**.

This is *not* a server security failure — it is a **browser security feature**.

2. Enabling CORS in ASP.NET Core

CORS must be configured in **two places**:

Step 1 — Define CORS Policy (ConfigureServices)

```
services.AddCors(options =>
{
    options.AddPolicy("AllowReactApp",
        builder => builder
            .AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader());
});
```

This allows:

- Any origin
- Any HTTP method
- Any header

(You may later restrict this in production.)

Step 2 — Apply Middleware (Correct Order Required)

```
app.UseRouting();  
  
app.UseCors("AllowReactApp"); // Must come here  
  
app.UseAuthorization();  
  
app.UseEndpoints(endpoints =>  
{  
    endpoints.MapControllers();  
});
```

Important Rule

If CORS is placed **after endpoints or authorization** → **CORS will NOT work.**

2. Model Binding Concepts (Very Important Theory)

Model Binding in ASP.NET Core **automatically maps incoming HTTP request data to action parameters / model objects.**

It reads value from:

- Form fields
- Route parameters
- Query string
- Headers
- Body (JSON)
- Uploaded files (via IFormFile)



Purpose

- ✓ Reduces boilerplate code
 - ✓ Enforces consistency
 - ✓ Automatically validates model contracts
-

3. Handling File Upload in Web API

File Upload Model — Single File

```
public class FileModel  
{  
    public IFormFile FormFile { get; set; }  
}
```

Where:

- IFormFile represents an uploaded file
- Only works when **Content-Type = multipart/form-data**

Multiple File Upload Model

```
public class FileModel
{
    public List<IFormFile> FormFiles { get; set; }
}
```

4. Creating the File Upload API Controller

Required Folder

Create a folder:

wwwroot

Files will be saved here.

Controller Pattern

```
[Route("api/[controller]")]
[ApiController]
public class FileController : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> Upload([FromForm] FileModel model)
    {
        var file = model.FormFile;

        if (file == null || file.Length == 0)
            return BadRequest("File not selected");

        var path = Path.Combine("wwwroot", file.FileName);

        using var stream = new FileStream(path, FileMode.Create);
        await file.CopyToAsync(stream);

        return Ok("File uploaded successfully");
    }
}
```

Key Concepts

- ✓ [FromForm] is **mandatory**
 - ✓ Request must be **multipart/form-data**
 - ✓ File is streamed to physical storage
 - ✓ Avoid reading entire file into memory
-

5. React — Upload File Using FormData

Browsers cannot send files in plain JSON.
Therefore, React uses **FormData**.

Why FormData?

Because it:

- ✓ Supports file + metadata in same request
 - ✓ Automatically sets Content-Type = multipart/form-data
-

Minimal React Upload Flow

```
const formData = new FormData();
formData.append("FormFile", selectedFile);

await axios.post(
  "https://localhost:5001/api/file",
  formData
);
```

Where "FormFile" must match **model property name**.

6. HTML Form Requirement

When using HTML form, always use:

```
<form enctype="multipart/form-data">
```

Otherwise file will not bind.

7. Common Issues & Root-Cause Analysis

Issue	Root Cause	Resolution
CORS Error	Policy missing / wrong order	Enable CORS before authorization
File is null	Wrong Content-Type	Use multipart/form-data
Large file upload fails	Request size limits	Increase request size
React cannot call API	Different origin blocked	Allow origin in CORS
Route not hit	Wrong endpoint/verb	Match endpoint + method

8. Increasing File Upload Size (If Required)

Example config:

```
services.Configure<FormOptions>(o =>
{
    o.MultipartBodyLengthLimit = long.MaxValue;
});
```

Only do this where justified.

9. Security Considerations (Professional Practice)

- ✓ Validate file type
- ✓ Sanitize filename
- ✓ Restrict file size
- ✓ Store uploads outside root in production
- ✓ Scan for malware where needed

Chapter 5 — ASP.NET Core MVC vs Web API, Middleware Pipeline, Dependency Injection & End-to-End Architecture

1. ASP.NET Core MVC vs Web API — Conceptual Clarity

Both **MVC** and **Web API** are built on **ASP.NET Core**.

They share:

- Same runtime
- Same pipeline
- Same configuration system
- Same dependency injection engine
- Same routing system

However, their **purpose and response behavior differ**.

Web API — Purpose

Used to build **HTTP services that return data**, primarily:

- JSON (default serializer: `System.Text.Json`)
- XML (if enabled)
- Binary streams
- Files



Typical consumers include:

- React, Angular, Vue
- Mobile apps (Android/iOS)
- IoT devices
- Other servers/microservices

No UI is returned — **only structured data**.

MVC — Purpose

MVC returns **HTML Views** in addition to JSON data.

Use MVC when:

- Server-side UI rendering is required
- Razor views are part of the application
- SEO-indexed web pages are needed

- SSR (server-rendered UI) is preferred

Key Functional Difference

Feature	Web API	MVC
Output	JSON/XML/Data	HTML Views + JSON
Consumer	Apps/Devices/APIs	Browsers + Clients
Decorator	[ApiController] often used	[Controller]
Validation	Automatic in API mode	Manual coding or model state check
UI Layer	None	Razor Views

2. ASP.NET Core Startup & Execution Pipeline

Every ASP.NET Core application starts as a **Console Program**.

Entry Point

Program.cs → Main()

The host is:

1. Created
2. Built
3. Run



Once running, it begins listening for HTTP requests.

3. Middleware — The Request Processing Pipeline

A **middleware** is a component that processes HTTP requests and responses.

Requests flow **in order**, like a chain.

Pipeline Structure Example

```
app.UseRouting();  
  
app.UseCors();  
  
app.UseAuthentication();  
  
app.UseAuthorization();  
  
app.UseEndpoints(...);
```

How Middleware Works (Conceptually)

A request passes through each middleware one-by-one:

1. Logging
 2. Exception handling
 3. Static files
 4. Routing
 5. Authentication
 6. Authorization
 7. Endpoint execution (controller runs)
 8. Response flows back outward through same pipeline
-

Important Property

ORDER MATTERS

Changing line order may break behavior — especially CORS, routing, auth.

4. Routing — How URLs Map to Controllers

Routing determines **which controller method executes**.

Common form:

```
[Route("api/[controller]")]
```

```
[ApiController]
```

```
public class EmployeeController : ControllerBase
```

```
{  
    [HttpGet("{id}")]  
    public IActionResult Get(int id)  
    {  
        ...  
    }  
}
```

URL Example:

/api/employee/10

Maps to:

Get(int id)

5. Dependency Injection (DI) — Core Concept of ASP.NET Core

ASP.NET Core is designed around **built-in dependency injection**.

Why DI?

Without DI:

- Classes create dependencies directly
 - Leads to tight-coupling
 - Hard to test
 - Hard to maintain
-

With DI

- Dependencies are **registered**
 - Injected automatically into constructors
-

Service Registration Example

```
builder.Services.AddScoped<IMembers, MembersRepository>();
```

Service Consumption

```
public class MemberController : ControllerBase
{
    private readonly IMembers member;

    public MemberController(IMembers mem)
    {
        member = mem;
    }
}
```

Lifetime Types

Lifetime	Meaning	Use When
Transient	New instance every request	Stateless services
Scoped	One instance per HTTP request	DB contexts, repositories
Singleton	One instance for app lifetime	Config, caching services

6. MVC Request-Handling Flow (High-Level)

1. Browser sends HTTP request
 2. Request goes through middleware pipeline
 3. Routing identifies controller/action
 4. Model binding maps request → parameters
 5. Filters execute (if any)
 6. Controller executes logic
 7. View engine processes Razor page (if MVC UI)
 8. Response returned to client
-

7. Web API Request-Handling Flow (High-Level)

1. Client sends HTTP request (JSON, params, headers)
 2. Request enters middleware pipeline
 3. Routing selects API controller action
 4. Model binding maps input to model/params
 5. Action executes business logic
 6. Action returns:
 - JSON data
 - Status code
 - Headers
 7. Response sent to client
-



8. Model Binding — Internal Understanding

Model binding automatically fills:

- Action parameters
- Model objects

From sources like:

- Query string
- Route values
- Headers
- Form values
- JSON body

- File uploads

This eliminates manual parsing.

9. Validation Behavior — Special in API Mode

When [ApiController] is applied:

- **Model validation happens automatically**
- Invalid model returns **HTTP 400 Bad Request**
- No need to manually check ModelState.IsValid

This is API-specific behavior.

10. JSON Serialization — Default Behavior

ASP.NET Core Web API uses:

System.Text.Json

Advantages:

- High-performance
- Lightweight
- Built-in

(You can still switch to Newtonsoft.Json if required.)

11. Real-World Architecture Alignment

Most production systems follow a **layered architecture**:

Client (UI / Mobile / React)

↓

Controller Layer (Web API)

↓

Business / Service Layer

↓

Repository / Data Layer

↓

Database

Why Layering?

- ✓ Loose coupling
- ✓ Maintainability
- ✓ Unit-test readiness
- ✓ Re-usability
- ✓ Clear responsibility separation

12. When to Choose MVC vs Web API

Choose MVC When

- You are building **server-rendered HTML UI**
- SEO matters
- You want Razor pages
- Minimal JavaScript usage

Choose Web API When

- The backend is **only data-service**
- UI exists separately (React/Angular/Mobile)
- Microservices architecture
- Third-party integrations
- IoT / headless systems

13. Exam-Level & Interview-Level Key Points

- ASP.NET Core Web API returns **JSON by default**
 - [ApiController] enables:
 - Auto validation
 - Attribute routing requirement
 - Better binding behavior
 - **201 Created must return Location header**
 - DI is **built-in**
 - Middleware runs **in sequence**
 - CORS must be placed **before authorization**
 - Route constraints prevent **ambiguity**
 - ActionResult is **preferred return type**
 - ASP.NET Core is **cross-platform and open-source**
-

Chapter 6 — Database Integration, Entity Framework Core, Repository Pattern & CRUD Flow in ASP.NET Core Web API

1. Role of Entity Framework Core (EF Core)

Entity Framework Core is Microsoft's **Object-Relational Mapper (ORM)** for .NET Core. It allows developers to work with relational databases using **C# classes instead of SQL queries**.

EF Core maps:

Class → Table

Property → Column

Object → Row

It eliminates most manual SQL CRUD coding.

2. Why Use EF Core

Key benefits:

- ✓ Strongly typed LINQ-based database operations
 - ✓ Database-first and code-first support
 - ✓ Built-in change tracking
 - ✓ Works across SQL Server, MySQL, PostgreSQL, SQLite, Oracle, etc.
 - ✓ Integrates natively with ASP.NET Core & Dependency Injection
-

3. Typical Architecture with EF Core

Controller → Service/Business Layer → Repository → EF DbContext → Database

This separation ensures:

- Loose coupling
 - Testability
 - Maintainability
 - Better layering
-

4. Model / Entity Class Example

```
public class Members
{
    public int MemberId { get; set; }

    public string FirstName { get; set; }
```

```
public string LastName { get; set; }

public string Address { get; set; }

}
```

EF Core maps this class to a **Members table**.

5. Defining DbContext

DbContext is the **bridge between application and database**.

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Members> Members { get; set; }
}
```

DbSet<T> represents a database table.

6. Registering DbContext in Dependency Injection

Example registration:

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection"));
```

Meaning:

- Connection string stored in configuration
 - EF Core will use SQL Server
 - AppDbContext is injected wherever needed
-

7. Repository Pattern — Business Contract Layer

Repository isolates **data access logic**.

Repository Interface

```
public interface IMembers
```

```

{
    List<Members> GetAllMember();

    Members GetMember(int id);

    Members Add(Members member);

    Members Update(int id, Members memberChanges);

    Members Delete(int id);
}

```

This acts as a **contract**.

8. Implementing Repository

```

public class MembersRepository : IMembers
{
    private readonly AppDbContext context;

    public MembersRepository(AppDbContext ctx)
    {
        context = ctx;
    }

    public List<Members> GetAllMember()
    {
        return context.Members.ToList();
    }

    public Members GetMember(int id)
    {
        return context.Members.Find(id);
    }

    public Members Add(Members member)
    {
        context.Members.Add(member);
        context.SaveChanges();
        return member;
    }

    public Members Update(int id, Members memberChanges)
    {
        var member = context.Members.Find(id);
        if (member == null) return null;

        member.FirstName = memberChanges.FirstName;
        member.LastName = memberChanges.LastName;
        member.Address = memberChanges.Address;
    }
}

```



```

    context.SaveChanges();
    return member;
}

public Members Delete(int id)
{
    var member = context.Members.Find(id);
    if (member == null) return null;

    context.Members.Remove(member);
    context.SaveChanges();

    return member;
}
}

```

9. Register Repository in DI

```
builder.Services.AddScoped<IMembers, MembersRepository>();
```

This ensures a **new instance per HTTP request**.

10. CRUD Flow Inside Web API Controller

Controller receives HTTP request → calls repository → returns response.

```

[Route("api/[controller]")]
[ApiController]
public class MemberController : ControllerBase
{
    private readonly IMembers member;

    public MemberController(IMembers mem)
    {
        member = mem;
    }

    [HttpGet]
    public IEnumerable<Members> Get() =>
        member.GetAllMember();

    [HttpGet("{id}")]
    public Members Get(int id) =>
        member.GetMember(id);

    [HttpPost]
    public Members Post([FromBody] Members memberpost) =>
        member.Add(memberpost);
}

```

```

[HttpPut("{id}")]
public Members Put(int id, [FromBody] Members memberput) =>
    member.Update(id, memberput);

[HttpDelete("{id}")]
public void Delete(int id) =>
    member.Delete(id);
}

```

11. Complete CRUD Request Examples

Action	HTTP Method	URL	Request Body
Get all	GET	/api/member	—
Get by ID	GET	/api/member/1	—
Create	POST	/api/member	JSON
Update	PUT	/api/member/1	JSON
Delete	DELETE	/api/member/1	—

12. Model Binding in CRUD

ASP.NET Core automatically maps:

- Route → id
- JSON body → Members object

Examples:

```

[HttpPut("{id}")]
public Members Put(int id, [FromBody] Members memberput)

```

13. Transaction & SaveChanges()

SaveChanges() commits changes to the database.

EF Core internally:

- Tracks entity state
 - Generates required SQL
 - Executes DB command
 - Commits transaction
-

14. Error Handling Best Practices

Recommended enhancements:

- ✓ Return ActionResult instead of direct entity
- ✓ Validate input
- ✓ Check null return values
- ✓ Return appropriate HTTP status

Example:

```
[HttpGet("{id}")]
public ActionResult<Members> Get(int id)
{
    var data = member.GetMember(id);
    if (data == null)
        return NotFound();

    return Ok(data);
}
```



Chapter 7 — Advanced REST Best Practices, Status Codes, Versioning, Security & Production Considerations in ASP.NET Core Web API

1. RESTful API Design — Core Principles

A well-designed REST API must be:

- ✓ **Resource-oriented**
- ✓ **Stateless**
- ✓ **Consistent in response format**
- ✓ **Standards-compliant with HTTP semantics**
- ✓ **Clear about errors and success responses**

Resource Naming Conventions

Always use **plural nouns**:

Purpose	Good	Avoid
Get all	/api/members	/api/getAllMembers
Get one	/api/members/5	/api/getMember?id=5
Create	/api/members	/api/createMember
Update	/api/members/5	/api/updateMember
Delete	/api/members/5	/api/deleteMember

2. Correct HTTP Status Code Usage

Clients must always know **what happened** — not just receive success/failure.

Common HTTP Status Codes

Code	Meaning	When to Use
200 OK	Success with response body	GET success
201 Created	Resource created	POST success
204 No Content	Success without response	DELETE success
400 Bad Request	Client input invalid	Model validation failed
401 Unauthorized	Auth required	Missing/invalid token
403 Forbidden	Access denied	User not allowed
404 Not Found	Resource missing	Record not found
409 Conflict	Data conflict	Duplicate unique value
500 Internal Server Error	Server failure	Exceptions

3. Correct POST Behavior — 201 Created

When a resource is created:

- ✓ Return **201 Created**
- ✓ Include **Location header**
- ✓ Return created object

Do NOT return 200 OK for POST unless required intentionally.

Already covered patterns:

- Created()
- CreatedAtAction()
- CreatedAtRoute()

These also help **self-discoverability** for clients.

4. Idempotency — PUT vs POST

Method	Meaning	Idempotent?	Usage
POST	Create resource	No	Insert
PUT	Replace resource	Yes	Update/Upsert
PATCH	Partial update	No*	Update fields

PUT should **fully replace** an entity.

PATCH updates only selected attributes.

5. Input Validation — API Behavior

When using [ApiController]:

- Invalid model state automatically returns **400 Bad Request**

Example triggers include:

- ✓ Missing required fields
- ✓ Invalid formats
- ✓ Constraint violations

This ensures **consistent contract enforcement**.

6. Exception Handling — Production-Grade

You should not expose exceptions to clients.

Recommended:

- Use **Global Exception Middleware**

- Log errors
- Return **generic structured error responses**

Structured JSON Example:

```
{
  "traceId": "abc123",
  "message": "An unexpected error occurred.",
  "status": 500
}
```

Purpose:

- ✓ Prevents leaking stack trace
- ✓ Creates audit trail
- ✓ Maintains client-friendly messages

7. CORS — Best-Practice Guidelines

Allowing **any origin** is acceptable in development, but in production **restrict origins**.

Example secure approach:

- Allow only specific domains
- Permit only necessary headers/methods
- Audit usage



8. Authentication & Authorization — Security Layer

Security is **mandatory** in professional APIs.

Common Approaches

- ✓ JWT Token authentication
- ✓ Identity Server / OAuth2
- ✓ Azure AD / enterprise identity
- ✓ Role-based authorization

Controller-level example pattern:

[Authorize]

```
public class MemberController : ControllerBase
```

```
{
}
```

Combine with:

- HTTPS enforcement
 - Token validation
 - Claims-based access
-

9. API Versioning — Stability & Evolution

Never break existing clients.

Use versioning when evolving APIs.

Common Versioning Patterns

Style	Example
URL Versioning	/api/v1/members
Header Versioning	api-version: 1.0

Versioning allows:

- ✓ Backward compatibility
 - ✓ Controlled upgrades
 - ✓ Parallel support
-

10. Logging — Observability Requirement

Logging is used for:

- Troubleshooting
- Monitoring usage
- Security audits
- API analytics

Use:

- ✓ Built-in logging
- ✓ Serilog
- ✓ ELK stack / Seq / App Insights

Always log:

- Request path
 - Status
 - Errors
 - User identity
 - Timestamp
-

11. Performance Considerations

Caching

Caching improves:

- ✓ Response speed
- ✓ Server efficiency

Common caching methods:

- In-memory
- Distributed cache (Redis)
- Response caching
- Output caching

Asynchronous Programming

Use async calls to:

- ✓ Improve scalability
- ✓ Prevent thread blocking

Pagination

Never return huge datasets without paging.

Use:

/api/members?page=2&pageSize=20

12. Deployment — Hosting Models

ASP.NET Core runs inside:

- ✓ IIS
- ✓ Linux (Nginx/Apache reverse proxy)
- ✓ Kestrel
- ✓ Docker containers
- ✓ Kubernetes & Cloud platforms

It is platform-independent.

13. Security Best Practices — Industry Standard

To protect APIs:

- ✓ Enforce HTTPS
 - ✓ Validate input
 - ✓ Apply rate limiting
 - ✓ Validate JWT tokens
 - ✓ Avoid exposing internal errors
 - ✓ Restrict CORS
 - ✓ Sanitize uploaded files
 - ✓ Encrypt sensitive data
-

14. Real-World Web API Checklist

Before publishing:

1. **Return correct HTTP codes**
2. **Protect endpoints with authentication**
3. **Validate all inputs**
4. **Log errors**
5. **Implement exception middleware**
6. **Secure CORS policy**
7. **Use HTTPS**
8. **Disable stack trace exposure**
9. **Apply rate limits (where needed)**
10. **Version your APIs**
11. **Use ActionResult consistently**
12. **Document your API (Swagger/OpenAPI)**



15. Swagger / OpenAPI — Documentation

Swagger helps:

- ✓ API discoverability
- ✓ Testing
- ✓ Client integration
- ✓ Contract clarity

It generates:

- UI documentation
 - Machine-readable OpenAPI specification
 - Client SDKs
-